# Local LLM on OpenVINO

## with ORBIT-LLM, an automatic configuration controller

Software Domain Project · Problem statement **SWFW_5S_13 — Local LLM on OpenVINO**
OpenVINO GenAI 2026.3, C++ API · Intel Core i5-1335U with Iris Xe iGPU, 15.7 GB RAM
November 2026

**Contents**

1. Project overview: what was built and the main results
2. Full report, organised by the four learning outcomes: deployment; inference, quantization and the KV cache; hardware; KV-cache management
3. Extension: ORBIT-LLM and its pre-registered evaluation on real request sizes
4. Limitations and how to reproduce

The detailed lab notebook (Findings 1–30), all source code, raw results and the bug reproductions are in the accompanying files of this submission.

# Local LLM on OpenVINO — with ORBIT-LLM

A 1–3B parameter language model running entirely on a laptop through the **OpenVINO GenAI C++ API**. Each part was measured: inference, quantization, the KV cache, hardware use and OpenVINO's KV-cache management. This is semester project **SWFW_5S_13 — Local LLM on OpenVINO** (Zoho).

Everything lives in `orbit-llm/`:

| Start here | What it is |
|---|---|
| `orbit-llm/REPORT.md` | The report, organized by the task's four learning outcomes |
| `orbit-llm/cpp/orbit_chat.cpp` | **The deliverable:** a local multi-turn chat app on the C++ API, with streaming, CPU or iGPU, KV-cache options and per-turn metrics |
| `orbit-llm/FINDINGS.md` | Lab log: every experiment with its method, raw-data path and conclusion |
| `orbit-llm/README.md` | Component map, build and demo commands |

## Results

Every speed and memory number comes from one laptop: Intel Core i5-1335U (2 P-cores + 8 E-cores), Iris Xe iGPU, 16 GB RAM, OpenVINO 2026.3. Models: TinyLlama-1.1B-Chat (FP16, INT8, INT4) and Qwen2.5-Coder-3B-Instruct (INT4). The HumanEval quality scores were run on a Colab CPU, so that all three 3B precisions ran on the same machine.

- **Quantization, speed (TinyLlama, CPU):** INT8 decodes 1.8–2.0× faster than FP16 and INT4 2.2–3.1× faster, using 48% and 67% less memory.
- **Quantization, quality (Qwen2.5-Coder-3B, HumanEval, 164 problems):**
- FP16 and INT8 each pass 137; INT4 passes 135.
- There is no detectable difference (McNemar p = 0.79), and INT4 weights are 71% smaller.
- A 95% interval still allows INT4 to be up to 5.7 points worse.
- TinyLlama-1.1B passes only 10–12 of 164 at every precision, so here model size limits quality far more than precision does.
- **Prefix caching:** a repeated 1,920-token prompt reaches its first token about 177× faster.
- **What the prefix cache keeps:**
- OpenVINO's default KV pool keeps only the latest prompt.
- A rule derived from the GenAI source predicts how many conversations a fixed pool keeps: K × [tokens / block size] blocks for K conversations.
- Measured: 45 of 45 hits on the iGPU and 23 of 24 on CPU at exactly that size, and 0 hits one block short.
- **KV-cache precision on the iGPU:**
- Read from the compiled model, OpenVINO stores the KV cache in **4 bits by default**: 11.5 KiB per token for the 3B model, against 36 at f16.
- Measured GPU memory matches that layout to within 1.5 MiB, and speed does not change.

- The 4-bit cache changes 40% of HumanEval answers (int8 only 5%). Retrieval at up to 3.5k tokens stays perfect.
- See Finding 30.
- **Hardware:** the iGPU is 3–7× faster than the CPU to the first token and 1.4–2.6× faster at generating. On the hybrid CPU, two P-cores beat the 12-thread default at generating by 16–40%.
- **Two OpenVINO bugs, reproduced minimally:**
- The GPU model cache silently ignores `KV_CACHE_PRECISION`: a later compile gets the first compile's KV layout. This reproduces on 2026.3 and 2026.4.1. Report text: `orbit-llm/notes/issue_gpu_cache_kv_precision_TO_FILE.md`.
- Combining cache eviction with prefix caching crashes a multi-turn chat and leaks KV blocks.
- A 60-line program reproduces it on GenAI 2026.3 and on the latest 2026.4.1.
- Report text: `orbit-llm/notes/issue_eviction_prefix_caching_TO_FILE.md`.

## Extension: ORBIT-LLM

ORBIT is a controller that predicts how every configuration (model, precision, device, cores) would perform for a request, picks the best one within the user's limits, and corrects itself from measurements. - **Prediction accuracy:** its predictor reaches within about 4 points of the run-to-run noise. - **Against fixed settings:** the first evaluation, on synthetic requests, showed **no win** over one well-chosen fixed configuration. - **On a real workload:** the second evaluation used 300 requests with real lengths, from the Azure 2023 LLM traces and OpenAssistant conversations, with its plan written down before any measurement. - ORBIT broke **84** service limits, against **96** for the best fixed configuration (exact McNemar p = 0.0005) and **152** for OpenVINO's default. - The gain comes from sending code to the 3B model, and a simple rule matches it (84). See Finding 29 and the plan.

## Quick start

Prerequisites: - Windows with Visual Studio 2022; - the Python packages `openvino==2026.3.0` and `openvino-genai==2026.3.0.0` in a virtual environment; - OpenVINO GenAI built from source in `openvino.genai/` (commit `bd8d6542`); - a model in OpenVINO IR format, e.g. `OpenVINO/Qwen2.5-Coder-3B-Instruct-int4-ov` from Hugging Face.

```
git clone --recurse-submodules https://github.com/parvatisanthosh/grape-wine.git
cd grape-wine/orbit-llm
cmake -S cpp -B cpp/build -G "Visual Studio 17 2022" -A x64 `
      -DOpenVINO_DIR="<venv>/Lib/site-packages/openvino/cmake"
cmake --build cpp/build --config Release
$env:Path = "<venv>\Lib\site-packages\openvino\libs;$env:Path"
.\cpp\build\Release\orbit_chat.exe --model <model-dir> --device GPU
```

Build details and every option: `orbit-llm/cpp/README.md`.

## Repository layout

| Path | Contents |
|---|---|
| `orbit-llm/` | The project: C++ programs, experiments, results, report |

| Path | Contents |
| --- | --- |
| `openvino.genai/` | OpenVINO GenAI source at commit `bd8d6542`, the version this project is built against |
| `LOG.md`, `test*.py`, `runchat.ps1` | First exploration with the Python API, before the C++ work |

# Local LLM on OpenVINO — Project Report

**Task:** SWFW_5S_13, *Local LLM on OpenVINO* — run a tiny 1B–3B parameter large language model entirely locally using the OpenVINO GenAI C++ API.

**Learning outcomes covered:**

1. An LLM deployed on a local machine with an OpenVINO backend
2. Understanding inference, quantization and the KV cache
3. Hardware requirements, utilization and performance factors
4. KV-cache management in OpenVINO

**Machine:** Intel Core i5-1335U (2 performance + 8 efficiency cores, 12 threads), Intel Iris Xe integrated GPU, 16 GB RAM, Windows 11. **Software:** OpenVINO 2026.3 and OpenVINO GenAI 2026.3 (built from source). **Models:**

| Model | Precision | Size on disk |
|---|---|---|
| TinyLlama-1.1B-Chat | FP16 | 2.1 GB |
| TinyLlama-1.1B-Chat | INT8 | 1.05 GB |
| TinyLlama-1.1B-Chat | INT4 | 0.62 GB |
| Qwen2.5-Coder-3B-Instruct | INT4 | 1.8 GB |

Every number in this report was measured on this machine. The full lab log, with methodology, raw-data paths and the reasoning behind each result, is **FINDINGS.md** (Findings 1–30). This report summarizes it by learning outcome.

---

## Summary

| Question | Answer, from measurements |
|---|---|
| Can a 1–3B LLM run fully locally through the GenAI C++ API? | Yes. `orbit_chat` is a multi-turn chat app built only on the C++ API. TinyLlama INT4 answers at ~20 tok/s on CPU and ~35 tok/s on the iGPU. |
| What does quantization buy? | For TinyLlama, INT8 decodes 1.8–2.0× faster than FP16 and INT4 2.2–3.1× faster, using 48% and 67% less memory, at no measurable (INT8) or 1.4% (INT4) quality cost. |
| What limits speed? | Prompt processing (prefill) is compute-bound; token generation (decode) is memory-bandwidth-bound. Both are much faster on the iGPU (3–7× TTFT, 1.4–2.6× decode). |

| Question | Answer, from measurements |
|---|---|
| What does the KV cache cost? | Exactly its theoretical size (11 KiB/token at u8 for TinyLlama). It is small next to the weights at 1–3B. Prompt-length activations grow memory ~12× faster. |
| Best KV-cache feature? | Prefix caching: a repeated 1920-token prompt reaches its first token ~177× faster (13 s → 75 ms). In chat, late turns are 12–23× faster with it, and use less memory. But the default pool keeps only the latest prompt; a fixed pool of K × ⌈tokens / block⌉ blocks keeps K (with one spare block each to be safe). Cache eviction gave no benefit in chat and crashes together with prefix caching. |
| Minimum hardware? | 1.1B INT4: ~1.3 GB RAM. 3B INT4: ~2.5 GB (iGPU) to ~4 GB (CPU) working set. On this laptop, only the iGPU makes 3B interactive. |
| Does automatic configuration help? | On 300 real-length requests, ORBIT broke 84 service limits, against 96 for the best fixed configuration ($p = 0.0005$) and 152 for OpenVINO's default. The gain comes from sending code to the 3B model, and a simple rule does equally well (84). |

---

# 1. Deployment: an LLM on the local machine with an OpenVINO backend

## What was built (all on the OpenVINO GenAI C++ API)

| Program | Purpose |
|---|---|
| `cpp/orbit_chat` | **The deployed application.** Interactive multi-turn chat with streamed output and the model's own chat template (`ov::genai::ChatHistory`). Choice of model, device (CPU/GPU) and CPU core type. Exposes the KV-cache controls: prefix caching, KV precision and cache eviction. Prints prompt size, TTFT, tok/s and memory after every turn. Can replay a scripted conversation for reproducible demos. |
| `cpp/orbit_run` | Measurement executor: one configuration per process, exact-length prompts, JSON metrics plus memory, CPU and power telemetry. Used for all experiments. |
| `cpp/orbit_worker` | Long-lived process that keeps several models loaded and serves requests over stdin/stdout. Used to measure residency, switching cost and cross-request cache reuse. |

## Running it

```
$env:Path = "C:\Users\parva\venv\Lib\site-packages\openvino\libs;$env:Path"

# Interactive chat on the CPU
.\cpp\build\Release\orbit_chat.exe --model C:\Users\parva\tinyllama-int4

# 3B model on the iGPU, with the compiled-model cache for fast start-up
```

```
.\cpp\build\Release\orbit_chat.exe --model C:\Users\parva\qwen2.5-coder-3b-int4 --device
GPU --cache-dir ov_cache\gpu

# Reproducible scripted demo, with per-turn metrics saved as JSON lines
.\cpp\build\Release\orbit_chat.exe --model C:\Users\parva\tinyllama-int4 `
    --script conversations\demo_short.txt --json-out demo.jsonl
```

Build instructions: cpp/README.md.

### Deployment lessons

- **Getting the runtime to load.** OpenVINO installed from pip does not put its DLLs where a C++ program finds them. The first C++ build exited silently (`STATUS_DLL_NOT_FOUND`). Fixes:
- put `openvino\libs` on `PATH`;
- copy `openvino_genai.dll` and `openvino_tokenizers.dll` next to the executable, because GenAI loads the tokenizer DLL from its own folder. The CMake build now does this.
- **Tokenizers differ.** Qwen2.5's exported tokenizer ignores `max_length`/`truncation`. Exact-length prompts are now built by slicing the token tensor (Finding 16).
- **Defaults matter.** `LLMPipeline` on CPU uses the paged-attention backend with prefix caching *on*. Passing a plain `SchedulerConfig{}` silently turns it off and caps prefill at 256 tokens per step. The apps start from the library's latency-oriented defaults (Finding 1).
- **C++ vs Python.** An interleaved A/B test found the C++ executor equal to or slightly faster than the Python API (INT8: −20% TTFT), and ~50 MB lighter, because there is no Python interpreter in the process.

---

## 2. Understanding inference, quantization and the KV cache

### Inference has two phases with different bottlenecks

- **Prefill** processes the whole prompt at once and produces the first token. Its latency is **TTFT** (time to first token). It is *compute-bound*: many tokens per weight load.
- **Decode** generates one token at a time. Its latency is **TPOT** (time per output token). It is *memory-bandwidth-bound*: every token streams all the weights once.

Evidence:

- TTFT grows roughly linearly with prompt length, and faster than linearly at long context, because attention cost grows with the square of the length. Qwen-3B on the GPU: 8.7 s at 2k tokens, 19.9 s at 4k, 53.2 s at 8k (Finding 18).
- TPOT barely moves with context length.
- Two P-cores decode *faster* than all 12 threads, by 16–40% (Finding 12). More threads add synchronization cost but no memory bandwidth, the resource decode actually needs.

## Quantization (TinyLlama-1.1B, CPU)

| | FP16 | INT8 | INT4 |
|---|---|---|---|
| Size on disk | 2098 MB | 1051 MB (−50%) | 624 MB (−70%) |
| Peak memory | ~4.1 GB | ~2.1 GB (−48%) | ~1.35 GB (−67%) |
| Decode speedup vs FP16 | 1× | 1.8–2.0× | 2.2–3.1× |

- **Decode speed tracks weight size**, as expected for a bandwidth-bound phase.
- **INT4 gains less than 2× over INT8** because ~12% of its layers stay at INT8.
- **"FP16" runs in f32 on this CPU.** The i5-1335U has no native FP16 compute (`INFERENCE_PRECISION_HINT = f32`), so FP16 weights are expanded to f32 at load: 1.1B × 4 bytes ≈ 4.1 GiB, exactly the measured footprint. On the iGPU, which has native FP16, the same model uses 2.5 GB and decodes 2.5–3.4× faster (Finding 11).
- **Quality cost is tiny.** `quality_eval.py` measures bits per byte on a fixed 34 KB local corpus of documentation and code. Lower is better, and it is comparable across tokenizers:

| Model | Bits per byte | vs FP16 |
|---|---|---|
| TinyLlama FP16 | 0.6922 | — |
| TinyLlama INT8 | 0.6922 | no measurable loss |
| TinyLlama INT4 | 0.7017 | 1.4% worse |
| Qwen2.5-3B INT4 | 0.5521 | 20% better than TinyLlama FP16 |

INT8 halves the memory at no measurable quality cost. INT4 saves two thirds for 1.4%. A larger quantized model beats a smaller full-precision one. - **Task-level check on the 3B model.** HumanEval runs each model's code against official tests on 164 programming problems. All three models ran greedily on one Colab CPU with the same execution settings, so only the weights differ:

| Qwen2.5-Coder-3B | Weights file | Problems passed | 95% CI |
|---|---|---|---|
| FP16 | 6.2 GB | 137/164 (83.5%) | 77.1–88.4% |
| INT8 | 3.1 GB | 137/164 (83.5%) | 77.1–88.4% |
| INT4 | 1.8 GB | 135/164 (82.3%) | 75.8–87.4% |

- INT4 has 71% smaller weights than FP16 and no detectable loss: 8 problems passed only on FP16 and 6 only on INT4 (McNemar p = 0.79).
- This does not prove zero loss. The 95% CI puts INT4's cost between −3.2 and +5.7 percentage points.
- INT8 gave the exact same answer as FP16 on 137 problems, but INT4 did on only 47. So INT4 changes the text often and rarely changes whether the code works.
- **TinyLlama-1.1B passes 10, 11 and 12 of 164 (6–7%)** at FP16, INT8 and INT4, on its own laptop files (Finding 28). At this size the model, not the precision, limits quality: most failures are real coding mistakes. A common one is naming a loop variable `I` and then using `i`.
- These pass rates depend on the prompt used, so they are not comparable with published HumanEval scores. Colab supplied quality numbers only; all speed and memory results in this report come from the laptop. The INT4 model is the same file on both machines.

### The KV cache

Each generated token attends to the keys and values of every earlier token. These are cached so they are not recomputed, so the cache grows linearly with context.

**Size per token** = 2 (K and V) × layers × KV heads × head dim × bytes per element:

| Model | Layers | KV heads | Head dim | Per token at u8 / f16 / f32 |
|---|---|---|---|---|
| TinyLlama | 22 | 4 | 64 | 11 / 22 / 44 KiB |
| Qwen2.5-3B | 36 | 2 | 128 | 18 / 36 / 72 KiB |

**Measured vs predicted:** going from 256 to 1920 prompt tokens, f16 used 14 MiB more than u8 against 17.9 MiB predicted, and f32 used 50 MiB more against 53.6 MiB. For Qwen, f16 − u8 at 1088 tokens was 18 MiB against 19.1 MiB predicted (Findings 6 and 17).

**But the KV cache is not what dominates memory growth.** Total memory grows ~0.13–0.23 MiB per prompt token, ~12× the KV cache. The best explanation tested is prefill activation buffers: measured growth is a constant 2.3–2.4× of `(hidden + 2 × MLP width) × 4 bytes` in both models. An earlier hypothesis — that it is full-vocabulary logits — was refuted by a prediction made before the 3B run (Finding 16).

---

## 3. Hardware requirements, utilization and performance factors

### Memory requirements

| Model | Device | Peak working set | Notes |
|---|---|---|---|
| TinyLlama INT4 | CPU | ~1.3 GB | |

| Model | Device | Peak working set | Notes |
|---|---|---|---|
| TinyLlama INT8 | CPU | ~2.1 GB | |
| TinyLlama FP16 | CPU | ~4.1 GB | f32 expansion |
| TinyLlama FP16 | iGPU | ~2.5 GB | |
| Qwen2.5-3B INT4 | iGPU | 2.5–3.3 GB | 256 → 8k tokens |
| Qwen2.5-3B INT4 | CPU | 3.8–4.2 GB | close to free RAM on this laptop |

- **Working set is the wrong number for "will it fit?"** A pre-allocated 4 GB KV cache is fully *committed* but barely touched, so the working set grows by only +12 MiB. Commit briefly peaks at ~2× the cache size (8.4 GB).
- **The model file is resident twice.** For INT4 TinyLlama, committed memory is 850 MiB against a 1350 MiB working set: the memory-mapped model file plus the compiled copy (Finding 9).
- All tools therefore record committed memory as well as working set.

## CPU vs integrated GPU

| Workload | Best CPU configuration | Iris Xe iGPU |
|---|---|---|
| TinyLlama INT4, 1024-token prompt | 9.0 s TTFT, 47 ms TPOT | **1.7 s**, **28 ms** |
| Qwen-3B INT4, 1024-token prompt | 29.0 s TTFT, 7 tok/s | **4.3 s**, **18 tok/s** |
| TinyLlama FP16, 1024-token prompt | 27.3 s TTFT, 6 tok/s | **2.4 s**, **20 tok/s** |

The iGPU is 3–7× faster to the first token, 1.4–2.6× faster at decode, and far more consistent: ±10% vs up to 2× run-to-run spread on the CPU (Findings 11 and 17).

## Performance factors found

| Factor | Effect | Finding |
|---|---|---|
| Core type on a hybrid CPU | 2 P-cores beat the 12-thread default on decode by 16–40%. The OpenVINO default was never the best CPU configuration. | 12 |
| GPU kernel compilation | First GPU load takes 6–21 s. `ov::cache_dir` cuts later loads to 2–3 s. The same cache makes CPU loads ~5× *slower*. | 13 |

| Factor | Effect | Finding |
|---|---|---|
| Keeping the model resident | A long-lived process (`orbit_worker`) gave 1.65 s TTFT vs 2.45 s for a fresh process (INT4, GPU, 1024 tokens), and avoids a 3.5 s (1.1B) / 8.6 s (3B) reload per request. | residency |
| GPU contention | With a second LLM running on the iGPU, the GPU path slows 1.6–2.2× but still beats the CPU (2.5 s vs 16.5 s TTFT). CPU paths slow too, because the iGPU shares memory bandwidth. | GPU contention |
| Power source | On battery the same chat ran ~10× slower (TPOT ~450 ms vs ~46 ms). All tools now record AC/battery and refuse to benchmark on battery by default. | 20 |
| Background load and thermals | Run-to-run spread is ~0.8–1.2× of the median. Background CPU explains only part of it (r = 0.25). One 9.8-minute whole-machine stall was caught by telemetry and filtered out. | 10, 18 |

---

## 4. KV-cache management in OpenVINO

OpenVINO GenAI manages the KV cache with **paged attention**: the cache is split into fixed-size blocks (32 tokens on CPU, 16 on GPU), allocated on demand by a scheduler (`SchedulerConfig`). Each management feature was tested.

| Feature | How it is controlled | Measured effect |
|---|---|---|
| **Prefix caching** | `enable_prefix_caching`; on by default in `LLMPipeline` | A repeated 1920-token prompt: TTFT 13 s → 72–77 ms (~177×). Unique prompts: no measurable cost (Finding 8). In chat, turn 2 of a conversation reached its first token in 0.3 s against 3.3 s for the cold first turn. |
| **Cache size** | `cache_size` (GB), `num_kv_blocks`, or dynamic | Pre-allocation is committed in full and peaks at ~2× during set-up, but is invisible in the working set (Finding 9). **The size also decides what prefix caching can keep:** the dynamic default keeps only the latest prompt. A fixed pool keeps exactly as many prompts as fit (Finding 26, below). |
| **KV precision** | `KV_CACHE_PRECISION` hint | CPU default: u8. **iGPU default: 4-bit** (the setting reads `dynamic`); u8 gives int8 and f16 works, f32 does not. Memory matches the exact layout within 1.5 MiB; speed is unaffected at ≤3.5k tokens. The GPU model cache silently ignores this setting. See below (Finding 30). |

| Feature | How it is controlled | Measured effect |
|---|---|---|
| **Attention backend** | `ATTENTION_BACKEND = PA` or `SDPA` | SDPA grew memory 2× faster with prompt length (+462 vs +222 MiB, 256 → 1920 tokens) and was 69% slower to first token. It is consistent with materializing the full attention-score matrix (Finding 7). |
| **Cache eviction** | `use_cache_eviction` + `CacheEvictionConfig(start, recent, max)` | No benefit in chat, and it crashes together with prefix caching. See below. |

## KV precision on the iGPU (Finding 30)

The precision OpenVINO actually uses was read from the compiled model, the same way GenAI sizes its KV blocks, not taken from the property.

| Qwen2.5-Coder-3B on the iGPU | 4-bit (default) | int8 | f16 |
|---|---|---|---|
| KV cache per token | 11.5 KiB | 20.5 KiB | 36.0 KiB |
| Extra GPU memory vs 4-bit at 3.5k tokens, measured (theory) | — | +31.6 MiB (+31.1) | +83.4 MiB (+84.7) |
| Time to first token at 3.5k tokens | 19.6 s | 18.7 s | 19.3 s |
| HumanEval answers identical to f16 | 99 / 164 | 156 / 164 | — |
| Long-context retrieval (1k–3.5k tokens) | 72 / 72 | 72 / 72 | 72 / 72 |

- **The iGPU quantizes the KV cache to 4 bits by default.** Every GPU result in this report used it. Keys are quantized per channel and values per token, with the scales stored inside each block.
- **At 1–3B parameters the saving is small:** 85 MiB at 3.5k tokens, about 3% of the process. There is no speed cost.
- **The 4-bit cache changes 40% of HumanEval answers** (int8: 5%). Retrieval at up to 3.5k tokens was perfect at every precision.
- Whether the changed answers are less often correct is not measured yet. The answers and a scoring notebook are included (Finding 30).
- **The model cache ignores the precision.** With `CACHE_DIR`, a later compile asking for another KV precision gets the first compile's blob, while the property reports the new value.
- Reproduced on OpenVINO 2026.3 and 2026.4.1 (`cpp/repro/repro_gpu_cache_kv_precision.py`), and not yet reported upstream.
- It likely explains October's GPU crash.
- To compare KV precisions on the GPU, compile without a model cache, or use one cache folder per precision.

## Prefix retention: what the pool keeps (Findings 24 and 26)

Prefix caching only helps if the earlier prompt is still in the KV pool. The OpenVINO source sets three rules: 1. The default **dynamic pool** grows only to fit the requests running now. 2. When a request needs blocks, it reuses cached blocks **least recently used first**. A prompt's first blocks are its oldest. 3. A cached prompt is matched **from its first block** and stops at the first missing one, so losing block 0 loses everything.

That predicts exactly how many conversations a fixed pool of N blocks can keep: K conversations of T tokens need K × ⌈T / block size⌉ blocks. The test sent K 1920-token conversations in turns through one TinyLlama INT4 pipeline (`prefix_pool_study.py`), each generating 16 tokens:

| Device (block size) | Blocks per conversation | K | Pool that keeps all K | One block less |
|---|---|---|---|---|
| iGPU (16) | 121 | 1, 2, 4, 8 | 121, 242, 484, 968: all hits | all misses |
| CPU (32) | 61 | 1, 2, 4 | 61, 122, 244: hits (1 miss in 24) | 60 fails to run; 121, 243: all misses |
| Either | — | 2 or more | Dynamic pool: all misses | — |

- **The prediction held on mains power**, for K = 1, 2, 4 and 8 on the iGPU and 1, 2 and 4 on CPU (`results/experiments/prefix_pool.jsonl`):

| Pool | iGPU hits | CPU hits |
|---|---|---|
| Exactly K × blocks per conversation | 45 / 45 | 23 / 24 |
| One block short | 0 / 42 | 0 / 12 |
| Default dynamic, K ≥ 2 | 0 / 42 | 0 / 12 |
| One or two spare blocks (CPU, K = 2) | — | 20 / 20 |

- **It is all or nothing.** Taking turns is the worst case for least-recently-used reuse, so one block short loses every conversation, not just one.
- **The exact boundary is not perfectly reliable on CPU:** 1 miss in 24. The likely cause, from the source, is that fresh blocks keep their creation time and ties are broken by hash order. That is an inference. Twenty runs with spare blocks had no miss, too few to prove the margin removes it. **A spare block per conversation costs 352 KiB and is cheap insurance.**
- **A hit is worth 170–260× in TTFT:** median 22 ms instead of 3.8 s on the iGPU, and 62 ms instead of 16 s on CPU.
- **Memory is cheap at this size.** At 11 KiB per token (u8, CPU), a 1920-token conversation is about 21 MiB. So a 256 MiB pool would keep about 12 of them.

- **Practical rule:** with prefix caching and more than one conversation per pipeline, set `num_kv_blocks` (or `cache_size`) to cover the conversations that should stay warm. The dynamic default keeps only one.
- A pool smaller than one prompt makes generation fail with an internal check (`GenerationStatus::IGNORED`), not a clear error message.

## Cache eviction study

Method: `orbit_chat` replays a 12-turn conversation (`conversations/long_recall.txt`).

- Turn 1 states two facts: project "ORBIT", lucky number 47.
- Turns 2–11 are unrelated questions, which push the conversation past the cache budget.
- Turn 12 asks for the facts back.

Runner: `run_chat_study.py`; raw rows: `results/chat_eviction.jsonl`. Budgets are start:recent:max tokens. All runs were on AC power and completed 12 turns.

**First result: the two features cannot be combined.** With prefix caching and eviction both on, the pipeline throws "*blocks across layers must be freed simultaneously*" once eviction begins: after 3 turns at `32:64:256`, and after 5 at `32:128:512`. This is documented in `notes/upstream_issue_eviction_prefix_caching.md`. Eviction was therefore tested with prefix caching off, and `orbit_chat` now turns prefix caching off when eviction is requested.

Measurements at turn 12, with a prompt of about 1000–1200 tokens:

| Model | Configuration | TTFT | Peak memory | Late-turn TPOT | Recalled the facts? |
|---|---|---|---|---|---|
| Qwen-3B, iGPU | **prefix caching on** | **0.45 s** | **2524 MiB** | 56 ms | yes |
| | prefix caching off | 5.4 s | 2604 MiB | 58 ms | yes |
| | eviction `32:128:512` | 5.0 s | 2847 MiB | 58 ms | yes |
| | eviction `32:64:256` | 5.2 s | 2844 MiB | 65 ms | yes |
| TinyLlama INT4, CPU | **prefix caching on** | **0.34 s** | **1337 MiB** | 53 ms | no |
| | prefix caching off | 7.8 s | 1494 MiB | 50 ms | no |
| | eviction `32:128:512` | 7.7 s | 1500 MiB | 50 ms | no |
| | eviction `32:64:256` | 7.7 s | 1502 MiB | 48 ms | no |

**Conclusions:**

- **For chat, keep prefix caching on and leave eviction off.** Prefix caching makes late turns 12–23× faster to first token and uses *less* memory, because only the new tokens need prefill activations.
- **Eviction gave no benefit here.** TTFT, TPOT and memory did not improve, and on the iGPU memory was ~240 MiB higher. Each turn still has to prefill the whole prompt, and eviction acts during generation; answers here are only 80 tokens, and the KV cache is small at this context length (~15–40 MiB). Eviction is aimed at long single generations, which this chat workload does not exercise.
- **Recall was not hurt.** Qwen recalled both facts under every budget. That does not prove the facts' tokens survived: the first answer token sees the whole prompt, and eviction keeps high-attention blocks. TinyLlama failed recall in every configuration, so its failure comes from the model, not from eviction.

---

## 5. Extension: ORBIT-LLM, an automatic configuration controller

The experiments show that the best configuration depends on the request (prompt length, repeated prefix), the machine (free RAM, contention, power) and the goal (speed, memory, quality). ORBIT-LLM automates the choice:

- **Layer 1 — Performance Twin** (`twin_v0.py`). Predicts load time, TTFT, TPOT and peak memory for each measured (model, device) candidate, using physical laws fitted to 557 runs.
- On held-out configurations it is within ~4 points of the run-to-run noise floor, and predicts memory to ~0.3%.
- It generalized to the 3B model and to prefix-cache hits, where gradient-boosted trees failed badly (Findings 15 and 19).
- **Layer 2 — Optimizer** (`orbit.py`). Drops candidates that would break the user's TTFT, total-time or memory limits, or risk running out of RAM. Picks by latency, memory or quality, and reports the Pareto front.
- **Layer 3 — Online correction.** After each request, per-candidate correction factors move toward the measured/predicted ratio.

**Preliminary evaluation** (`evaluate_controller.py`, through `orbit_worker`, 24 requests per system, including scenarios under GPU contention):

| **All 24 requests** | **ORBIT** | **Static best (INT4 @ iGPU)** |
|---|---|---|
| SLA violations | 10 (6 refused, 4 missed) | 10 (all missed) |

| **The 18 requests both systems served** | **ORBIT** | **Static best (INT4 @ iGPU)** |
|---|---|---|
| SLA violations | 4 | 4 |
| Mean TTFT | 1.75 s | 1.74 s |
| Mean total time | 6.5 s | 4.5 s |

| The 18 requests both systems served | ORBIT | Static best (INT4 @ iGPU) |
|---|---|---|
| Mean peak memory | 2687 MiB | 1093 MiB |
| Mean bits per byte (lower = better) | 0.677 | 0.702 |

An earlier version of this table compared ORBIT's mean TTFT on 18 requests with static's on 24, which made ORBIT look faster. `evaluate_controller.py` now compares systems only on requests they all served.

- **ORBIT did not beat the static configuration.** The two had the same outcome on 22 of 24 requests, because ORBIT chose INT4 on the iGPU for 15 of the 18 requests it served. The other 3 were quality-first requests, where it chose the 3B model: better quality, but more time and memory.
- **This workload cannot separate the two systems.** On this laptop one configuration dominates most requests, and more requests from the same scenarios would not change that. A fair test needs requests where the best choice differs.
- **Six of the ten violations were follow-ups the prefix cache should have served.** Finding 24 explains why it did not: in the default dynamic pool, any other prompt between two repeats pushes the earlier prompt out. ORBIT also refused those follow-ups, so it never warmed its own cache.

## Evaluation v2: a real workload, a pre-registered test (Finding 29)

The preliminary result led to a second evaluation, fixed in advance in notes/evaluation_v2_plan.md before any data existed. - **Workload:** 300 test requests with real lengths: chat turns from OpenAssistant (oasst1), and one-shot and code requests from the Azure 2023 LLM inference trace. - **Limits per class:** - chat and one-shot: first token within 2 s, and at least 10 tokens/s; - code: first token within 1.5 s, and a model that passes at least 50% of HumanEval. - **Systems:** five, all run on the laptop on mains power, on the same requests. - **Best fixed:** chosen on a separate tuning trace, then frozen.

| 300 test requests | SLA violations | 95% CI |
|---|---|---|
| OpenVINO default (TinyLlama INT8 @ CPU) | 152 | 45–56% |
| Best fixed, tuned (TinyLlama INT4 @ iGPU) | 96 | 27–38% |
| Simple rule (3B for code or long prompts, else TinyLlama INT4 @ iGPU) | 84 | 23–33% |
| ORBIT without online correction | 88 | 24–35% |
| **ORBIT** | **84** | 23–33% |

- **The primary test is significant.** ORBIT vs best fixed: 12 requests only best fixed broke, 0 only ORBIT broke (exact McNemar p = 0.0005).

- The gain is routing. ORBIT sends short code requests to the 3B model, which TinyLlama may not serve (HumanEval 7% vs 82–84%).
- On requests both served, ORBIT is exactly as fast (time ratio 0.997, 95% CI 0.988–1.005).
- **ORBIT beats the OpenVINO default by 68 violations** ($p < 10^{-20}$), and is 2.8× faster on the requests both served.
- **ORBIT ties the simple rule** (84 vs 84). On this laptop, choosing the model by request type captures the whole benefit, and the Twin's finer choices change nothing.
- **Online correction makes the Twin accurate, not the outcome better.** The median TTFT error falls from 30% to 14%, and the decode-speed error from 61% to 5%. The controller swaps models 26 times instead of 69, but has only 4 fewer violations ($p = 0.22$).
- **The noise floor** (best fixed run twice) is 1 violation and 1.6% in time. Smaller differences between passes are not claimed.
- **Most violations are limits this hardware cannot meet**, and fail for every system. The 3B model reaches a first token within 1.5 s only for code prompts up to about 300 tokens, while the median code prompt is 1,245 tokens.
- **The cost of routing is memory:** two models stay resident, about 3.5 GB at peak against 1.1 GB.
- **A measurement fault was found and corrected before reporting.**
- *What happened:* five long prompts were not unique, because the Qwen tokenizer cuts text over 32,768 tokens from the left. Their request tag was lost, and they hit the prefix cache.
- *Correction:* the prompt builder was fixed, the 9 affected rows were re-measured, and the change is recorded in the plan (section 13).
- *Effect:* the conclusions are unchanged (uncorrected: 14 to 0).

---

## 6. Limitations and future work

- **One machine.** Results are specific to a 15 W hybrid laptop CPU with an iGPU. The methodology and tools are portable; the numbers are not.
- **Quality is measured on code only.** Both models have a task-level score (HumanEval), but there is no task-level test of chat answers. For chat, quality rests on bits per byte, which measures how well a model predicts real text.
- **GPU memory accounting.** iGPU buffers live in shared system memory and are only partly visible in process memory counters.
- **KV-cache precision on the GPU** was measured on one 3B model at up to 3.5k tokens (Finding 30). Its effect at much longer contexts, where 4-bit error and memory savings both grow, is untested.
- **Cache eviction was tested on chat only**, with 80-token answers. Its intended use, very long single generations, was not tested.
- **ORBIT's measured win comes from routing, which a one-line rule also achieves** on this laptop (Finding 29).
- Conditions that change during a session, such as a busy iGPU or a switch to battery, are where a fixed rule cannot adapt. They were not part of the pre-registered test.
- Requests ran one at a time. Real arrival times and queueing were not modeled.
- **Chat prompt contents are synthetic.** Only their lengths and turn structure come from real data. A follow-up reuses the earlier prompt as a prefix, but not the earlier reply.
- **ORBIT's prefix-hit predictor handles exact repeats only,** not the partial reuse a chat follow-up gets. It errs on the slow side.

## 7. Reproducing

See README.md for the component map and quick start. Each experiment in `plans/` can be rerun with `python run_experiment.py plans/<name>.json`, analyzed with `python analyze_experiment.py results/experiments/<name>.jsonl`, and folded into the predictor with `python build_dataset.py && python twin_v0.py`.